

ONE VOICE TECH ACADEMY

ADVANCED POWER BI – DAX/MEASURES HANDOUT

CHAPTER 1

Introduction to DAX & Measures

1.1 What Is DAX?

DAX stands for Data Analysis Expressions.

It is the formula language used in Power BI to create calculations and analyze data.

If you have worked with Excel formulas, some DAX functions will look familiar:

SUM()

AVERAGE()

MIN()

MAX()

However, DAX is designed to work with tables, columns, relationships, and filters inside a Power BI data model.

1.2 Why Do We Use DAX?

Power BI can import and visualize data without DAX. However, DAX allows us to answer questions that require calculations.

For example:

- What is our total sales?
- What is our total profit?
- How many customers do we have?
- What is our average sales value?
- Which region generated the most sales?

- How much have sales increased compared with last year?

DAX helps turn raw data into useful business information.

1.3 What Is a Measure?

A measure is a calculation created using DAX.

For example, suppose we have a table called Sales containing:

Product	Region	Sales Amount
Laptop	North	\$1,700
Mouse	South	\$125
Monitor	North	\$660
Keyboard	West	\$180

We can create a measure to calculate total sales:

Total Sales =

SUM(Sales[Sales Amount])

The result is:

Total Sales = \$2,665

In this formula:

- Total Sales → Name of the measure
- SUM() → DAX function
- Sales → Table name
- [Sales Amount] → Column being calculated

1.4 How DAX Refers to Columns and Measures

A column is usually referenced using:

TableName[ColumnName]

Example:

Sales[Sales Amount]

A measure is referenced using its name inside square brackets:

[Total Sales]

This distinction becomes important when we begin creating more advanced calculations.

1.5 Measures Are Dynamic

One of the most important things to understand about measures is that their results can change according to the filters in a report.

Suppose:

Total Sales =

SUM(Sales[Sales Amount])

A Card visual might show:

Total Sales: \$850,000

Now place Region beside the same measure:

Region	Total Sales
North	\$180,000
South	\$290,000
East	\$215,000
West	\$165,000
Total	\$850,000

We did not create four different measures.

Power BI used the same Total Sales measure and calculated it for each region.

If a user selects only South from a slicer, the measure will return:

\$290,000

This ability to respond to filters is one of the main reasons measures are so useful.

Key Point: A measure stores a calculation, not one fixed answer. Its result can change depending on the current report context.

We will learn more about filter context in Chapter 4.

1.6 DAX vs Measures

DAX and measures are related, but they are not the same thing.

Think of it this way:

DAX is the language. A measure is a calculation written using that language.

For example:

Total Quantity =

SUM(Sales[Quantity])

The formula is written using DAX, while Total Quantity is the measure we created.

1.7 More Examples of Simple Measures

Total Sales

Total Sales =

SUM(Sales[Sales Amount])

Average Sales

Average Sales =

AVERAGE(Sales[Sales Amount])

Total Quantity

Total Quantity =

SUM(Sales[Quantity])

Highest Sale

Highest Sale =

MAX(Sales[Sales Amount])

Measures can then be added to Cards, Tables, Matrices, Charts, and other Power BI visuals.

Quick Check

Before moving on, make sure you can answer these questions:

1. What does DAX stand for?
2. What is DAX used for?
3. What is a measure?
4. Is DAX the same thing as a measure?
5. What does Sales[Sales Amount] represent?
6. Why can the same measure return different results when a report is filtered?

Chapter Summary

- DAX stands for Data Analysis Expressions.
- It is the formula language used to create calculations in Power BI.
- A measure is a calculation written using DAX.
- Columns are commonly referenced as Table[Column].
- Measures are commonly referenced as [Measure Name].
- Measures are dynamic and can respond to filters in a report.

- Both measures and calculated columns can use DAX, but they behave differently. Understanding when to use each is an important foundation for working with Power BI.

CHAPTER 2

Measures vs Calculated Columns

Both measures and calculated columns can be created using DAX, but they serve different purposes.

A simple way to remember the difference is:

Calculated Column → calculates a value for each row.

Measure → calculates a result based on the current report context.

2.1 What Is a Calculated Column?

A calculated column is a new column created using a DAX formula.

The calculation is performed row by row.

Suppose our Sales table contains:

Product	Quantity	Unit Price
Laptop	2	\$850
Mouse	5	\$25
Monitor	3	\$220

We can create:

Sales Amount =

Sales[Quantity] * Sales[Unit Price]

Power BI calculates the result for every row:

Product	Quantity	Unit Price	Sales Amount
Laptop	2	\$850	\$1,700
Mouse	5	\$25	\$125
Monitor	3	\$220	\$660

Each transaction now has its own Sales Amount.

2.2 What Is a Measure?

A measure calculates a result when it is needed in a report.

Using the Sales Amount column above, we can create:

Total Sales =

SUM(Sales[Sales Amount])

The measure returns:

Total Sales = \$2,485

Unlike a calculated column, Power BI does not create a new Total Sales value for every row.

Instead, the measure calculates the result according to the current report context.

For example:

Region	Total Sales
North	\$1,700
South	\$125
East	\$660
Total	\$2,485

The same measure automatically produces the appropriate result for each region.

2.3 The Main Difference

Calculated Column	Measure
Calculates row by row	Calculates a result
Creates a new column	Creates a reusable calculation
Values are stored in the data model	Result is calculated when needed
Useful for row-level information and categories	Useful for summaries and business metrics
Does not dynamically recalculate its stored row values because of report filters	Responds dynamically to report filters

Quick Rule: If you need a value for each row, think Calculated Column. If you need a result that should change with the report, think Measure.

2.4 When Should You Use a Calculated Column?

Use a calculated column when you need information for each individual row.

Example 1: Sales Amount

Sales Amount =

Sales[Quantity] * Sales[Unit Price]

Every transaction receives its own Sales Amount.

Example 2: Customer Category

Suppose we want to classify customers based on age:

Age Group =

IF(

```
Customers[Age] < 30,  
"Under 30",  
"30 and Above"  
)
```

Every customer receives a category.

Calculated columns are particularly useful when the result needs to be used as a category, slicer, filter, row, or chart axis.

2.5 When Should You Use a Measure?

Measures are usually the better choice for business metrics and summary calculations.

Examples include:

Total Sales =

SUM(Sales[Sales Amount])

Average Sales =

AVERAGE(Sales[Sales Amount])

Total Quantity =

SUM(Sales[Quantity])

Measures are ideal when the answer should change according to:

- Region
- Product
- Date
- Customer
- Slicers
- Other report filters

For example, one Total Sales measure can show total company sales on a Card and sales for each region in a chart.

You do not need to create separate measures for North Sales, South Sales, East Sales, and West Sales just to display sales by region.

2.6 A Common Beginner Mistake

A common mistake is creating a calculated column when the calculation should be a measure.

For example, if the goal is simply to display total company sales, we do not need a new column containing the total.

Create a measure:

Total Sales =

SUM(Sales[Sales Amount])

Similarly, calculations such as:

- Total Profit
- Average Sales
- Customer Count
- Number of Orders
- Profit Margin
- Sales Growth

will usually be created as measures.

2.7 Which One Should I Use?

Consider the question you are trying to answer.

Requirement	Best Choice
Sales amount for every transaction	Calculated Column
Total sales	Measure
Classify each customer by age	Calculated Column

Requirement	Best Choice
Average sales	Measure
Total profit	Measure
Label each product High/Low Price	Calculated Column
Number of unique customers	Measure
Profit margin	Measure

The goal is not to memorize the table.

Instead, ask:

Do I need a value for every row, or do I need a calculation that summarizes data and responds to my report?

Quick Exercise

Decide whether each should be a Measure or Calculated Column.

1. Calculate Quantity $\times$ Unit Price for every transaction.
2. Calculate total revenue.
3. Label every order as "Large" or "Small".
4. Calculate average revenue.
5. Calculate the number of customers.
6. Calculate total profit.

Answers

1. Calculated Column
2. Measure
3. Calculated Column

4. Measure
5. Measure
6. Measure

Chapter Summary

- Both measures and calculated columns can use DAX.
- A calculated column calculates a value for each row.
- A measure calculates a result based on the current report context.
- Calculated columns are useful for row-level calculations and categories.
- Measures are generally used for totals, averages, percentages, counts, and other business metrics.
- Measures respond dynamically to filters and slicers.

CHAPTER 3

Basic DAX Functions

DAX contains many functions, but you do not need to learn all of them at once.

This chapter covers some of the most commonly used functions for everyday Power BI analysis.

We will focus on:

SUM, AVERAGE, MIN, MAX, COUNT, COUNTA, COUNTROWS, DISTINCTCOUNT, and DIVIDE.

3.1 SUM

SUM adds all the numbers in a column.

Syntax

SUM(Table[Column])

Example

Total Sales =

SUM(Sales[Sales Amount])

If Sales Amount contains:

Sales Amount
\$1,700
\$125
\$660
\$180

Total Sales returns:

\$2,665

Another example:

Total Quantity =

SUM(Sales[Quantity])

3.2 AVERAGE

AVERAGE calculates the arithmetic mean of the numeric values in a column.

Example

Average Sales =

AVERAGE(Sales[Sales Amount])

If the sales values are:

100, 200, 300

the result is:

$$(100 + 200 + 300) \div 3 = 200$$

Therefore:

$$\text{Average Sales} = \$200$$

3.3 MIN and MAX

MIN returns the smallest value in a column.

Lowest Sale =

MIN(Sales[Sales Amount])

MAX returns the largest value.

Highest Sale =

MAX(Sales[Sales Amount])

For:

125, 180, 660, 1700

the results are:

Lowest Sale = 125

Highest Sale = 1700

3.4 COUNT and COUNTA

COUNT

COUNT counts the numeric values in a column.

Number of Recorded Quantities =

COUNT(Sales[Quantity])

If the column contains:

2

5

3

Blank

4

the result is:

4

The blank is not counted.

COUNTA

COUNTA counts non-blank values and can be used with text as well as numbers.

For example:

Products Recorded =

COUNTA(Sales[Product])

If Product contains:

Laptop

Mouse

Monitor

Blank

Keyboard

the result is:

4

Remember: COUNT is for numeric values, while COUNTA counts non-blank values.

3.5 COUNTROWS

COUNTROWS counts the number of rows in a table.

Syntax

COUNTROWS(Table)

Example:

Number of Transactions =

COUNTROWS(Sales)

If the Sales table contains 5,000 rows:

Number of Transactions = 5,000

This is useful when each row represents something meaningful, such as one transaction or one order.

3.6 DISTINCTCOUNT

Sometimes we do not want to count every occurrence.

We want to know how many unique values exist.

Suppose:

Order ID	Customer
1001	Ada
1002	James
1003	Ada
1004	Sarah
1005	James

There are 5 transactions but only 3 different customers.

We can use:

Unique Customers =

DISTINCTCOUNT(Sales[Customer])

Result:

3

This is different from counting the rows because Ada and James appear more than once.

Another common example

Total Orders =

DISTINCTCOUNT(Sales[Order ID])

This is particularly useful when an Order ID can appear on several rows because one order contains multiple products.

3.7 DIVIDE

Division can be written using /, but DAX also provides the DIVIDE function.

Suppose we already have:

Total Sales =

SUM(Sales[Sales Amount])

and:

Total Cost =

SUM(Sales[Cost])

We can calculate profit:

Total Profit =

[Total Sales] - [Total Cost]

Then calculate profit margin:

Profit Margin =

DIVIDE(

[Total Profit],

[Total Sales]

)

If:

Total Sales = \$100,000

Total Profit = \$25,000

then:

Profit Margin = 25%

Why use DIVIDE instead of /?

DIVIDE handles situations where the denominator is zero or blank more safely.

We can even provide an alternative result:

Profit Margin =

DIVIDE(

[Total Profit],

[Total Sales],

0

)

The 0 means:

If Total Sales cannot be used as a valid denominator, return 0 instead of an error.

3.8 Putting the Functions Together

With only the functions covered in this chapter, we can already create several useful measures.

Total Sales =

SUM(Sales[Sales Amount])

Average Sales =

AVERAGE(Sales[Sales Amount])

Highest Sale =

MAX(Sales[Sales Amount])

Lowest Sale =

MIN(Sales[Sales Amount])

Number of Transactions =

COUNTROWS(Sales)

Unique Customers =

DISTINCTCOUNT(Sales[Customer ID])

These measures can then be placed in Cards, tables, charts, and other Power BI visuals.

3.9 Choosing the Right Function

If you want to...	Use
Add numbers	SUM
Find the average	AVERAGE
Find the smallest value	MIN
Find the largest value	MAX
Count numeric values	COUNT
Count non-blank values	COUNTA
Count table rows	COUNTROWS
Count unique values	DISTINCTCOUNT
Divide values safely	DIVIDE

You do not need to memorize every DAX function.

The important skill is being able to identify what calculation you need and then select the appropriate function.

Quick Exercise

Assume the Sales table contains:

Order ID

Customer ID

Product

Quantity

Sales Amount

Cost

Write measures to calculate:

1. Total Sales
2. Average Sales
3. Highest Sale
4. Lowest Sale
5. Number of transactions
6. Number of unique customers

Answers

Total Sales =

SUM(Sales[Sales Amount])

Average Sales =

AVERAGE(Sales[Sales Amount])

Highest Sale =

MAX(Sales[Sales Amount])

Lowest Sale =

MIN(Sales[Sales Amount])

Number of Transactions =

COUNTROWS(Sales)

Unique Customers =

DISTINCTCOUNT(Sales[Customer ID])

Chapter Summary

The most common basic DAX functions include:

- SUM — adds values
- AVERAGE — calculates an average
- MIN — returns the smallest value
- MAX — returns the largest value
- COUNT — counts numeric values
- COUNTA — counts non-blank values
- COUNTROWS — counts rows
- DISTINCTCOUNT — counts unique values
- DIVIDE — performs safe division

These functions are simple, but they form the foundation for many useful Power BI measures.

CHAPTER 4

Understanding Row Context & Filter Context

One of the most important ideas in DAX is context.

Context simply means:

The conditions under which DAX performs a calculation.

Two important types are:

- Row Context
- Filter Context

Understanding these will make more advanced DAX much easier.

4.1 Row Context

Row context means that DAX is evaluating a calculation for the current row.

We commonly see this with calculated columns.

Suppose we have:

Product	Quantity	Unit Price
Laptop	2	\$850
Mouse	5	\$25
Monitor	3	\$220

We create a calculated column:

Sales Amount =

Sales[Quantity] * Sales[Unit Price]

Power BI calculates each row separately.

For the Laptop row:

$$2 \times \$850 = \$1,700$$

For the Mouse row:

$$5 \times \$25 = \$125$$

For the Monitor row:

$$3 \times \$220 = \$660$$

DAX knows which Quantity and Unit Price to use because it is evaluating the current row.

That is row context.

Think: Row Context = “Which row am I currently on?”

4.2 Filter Context

Filter context refers to the filters that determine which data is included in a calculation.

Suppose we create:

Total Sales =

SUM(Sales[Sales Amount])

The complete dataset may produce:

Total Sales = \$850,000

Now suppose we place Region and [Total Sales] in a table:

Region	Total Sales
North	\$180,000
South	\$290,000
East	\$215,000
West	\$165,000
Total	\$850,000

The formula did not change.

Power BI evaluated it under a different filter context for each row of the visual.

For North, the calculation effectively becomes:

Calculate Total Sales using only data for North.

For South:

Calculate Total Sales using only data for South.

This is filter context.

4.3 Where Do Filters Come From?

Filter context can come from different parts of a Power BI report.

Slicers

If a user selects:

Year = 2026

the measures on the report are calculated using the filtered data for 2026.

Visuals

If Product is placed on a chart axis, Power BI calculates the measure separately for each product.

Report and Page Filters

Filters applied to a page or the entire report also affect measures.

Other Visuals

Selecting an item in one visual can filter or highlight data in another visual.

4.4 One Measure, Many Results

Consider:

Total Sales =

SUM(Sales[Sales Amount])

The same measure could show:

Card

Total Sales

\$850,000

Sales by Region

Region	Total Sales
North	\$180,000
South	\$290,000
East	\$215,000
West	\$165,000

Sales by Product

Product	Total Sales
Laptop	\$350,000
Monitor	\$210,000
Keyboard	\$165,000
Mouse	\$125,000

We do not need separate measures such as:

North Sales

South Sales

Laptop Sales

Monitor Sales

Power BI uses the existing filter context to determine what [Total Sales] should return.

This is one of the major advantages of measures.

4.5 Row Context vs Filter Context

Row Context	Filter Context
Works with the current row	Works with a filtered set of data
Common in calculated columns	Very important for measures
Evaluates data row by row	Changes according to filters
Think “this row”	Think “which data is currently included?”

A simple way to remember them:

Row Context = Which row?

Filter Context = Which data?

4.6 A Simple Example

Suppose our data contains:

Product	Region	Quantity	Unit Price
Laptop	North	2	\$850
Mouse	South	5	\$25
Monitor	North	3	\$220
Keyboard	West	4	\$45

First, we create a calculated column:

Sales Amount =

Sales[Quantity] * Sales[Unit Price]

This uses row context because Power BI performs the calculation for each row.

Next, we create:

Total Sales =

SUM(Sales[Sales Amount])

If the report is filtered to:

Region = North

the measure uses filter context and calculates only the North transactions:

$\$1,700 + \$660 = \$2,360$

Therefore:

Total Sales = \$2,360

4.7 Why Context Matters

A DAX formula cannot always be understood by looking at the formula alone.

Consider:

Total Sales =

SUM(Sales[Sales Amount])

The answer could be:

\$850,000

or:

\$290,000

or:

\$45,000

All three could be correct.

The result depends on the filter context in which the measure is evaluated.

This is why understanding context is more important than simply memorizing DAX functions.

Quick Check

Question 1

A calculated column calculates:

`Sales[Quantity] * Sales[Unit Price]`

for every transaction.

Which context is involved?

Answer: Row Context

Question 2

A slicer is changed from 2025 to 2026, and `[Total Sales]` changes automatically.

Which context explains this?

Answer: Filter Context

Question 3

The same `[Total Sales]` measure displays a different value for every region in a chart.

Why?

Answer: Each region creates a different filter context for the measure.

Chapter Summary

- Context describes the conditions under which a DAX calculation is evaluated.
- Row context focuses on the current row.
- It is commonly encountered in calculated columns.
- Filter context determines which data is included in a calculation.
- Slicers, visuals, and report filters can create filter context.
- Measures automatically respond to changes in filter context.

A simple way to remember:

Row Context = Which row?

Filter Context = Which data?

CHAPTER 5

CALCULATE and Filters

CALCULATE is one of the most important functions in DAX.

In simple terms:

CALCULATE evaluates a calculation using a specified or modified filter context.

It allows us to take an existing calculation and ask more specific questions.

5.1 Understanding CALCULATE

Suppose we already have:

Total Sales =

SUM(Sales[Sales Amount])

This calculates total sales according to the current report context.

But what if we specifically want sales from the North region?

We can write:

North Sales =

```
CALCULATE(  
    [Total Sales],  
    Sales[Region] = "North"  
)
```

This means:

Calculate [Total Sales], but apply North as the Region filter.

5.2 CALCULATE Syntax

The basic structure is:

```
CALCULATE(  
    Expression,  
    Filter  
)
```

The expression is what we want to calculate.

The filter tells Power BI the context in which we want the calculation performed.

For example:

Laptop Sales =

```
CALCULATE(  
    [Total Sales],  
    Sales[Product] = "Laptop"  
)
```

Here:

- [Total Sales] → Expression
- Sales[Product] = "Laptop" → Filter

The result contains sales for Laptop only.

5.3 Using Multiple Filters

CALCULATE can apply more than one filter.

For example:

North Laptop Sales =

```
CALCULATE(  
    [Total Sales],  
    Sales[Product] = "Laptop",  
    Sales[Region] = "North"
```

```
[Total Sales],  
Sales[Region] = "North",  
Sales[Product] = "Laptop"  
)
```

This calculates sales where:

Region = North

AND

Product = Laptop

So CALCULATE allows us to make our measures more specific.

5.4 CALCULATE With Existing Report Filters

Suppose a user selects:

Year = 2026

and we have:

Laptop Sales =

```
CALCULATE(  
    [Total Sales],  
    Sales[Product] = "Laptop"  
)
```

Power BI can combine these conditions.

The measure effectively answers:

What were Laptop sales in 2026?

This is why understanding filter context from Chapter 4 is important before learning CALCULATE.

5.5 FILTER

Sometimes the filtering condition we need is more complex.

For this, DAX provides the FILTER function.

Suppose we want sales from transactions worth more than \$1,000.

High Value Sales =

```
CALCULATE(  
    [Total Sales],  
    FILTER(  
        Sales,  
        Sales[Sales Amount] > 1000  
    )  
)
```

FILTER goes through the specified table and keeps only rows that satisfy the condition.

In this example:

Keep rows where Sales Amount is greater than \$1,000, then calculate Total Sales.

5.6 When Do We Need FILTER?

For simple conditions, we can usually write:

North Sales =

```
CALCULATE(  
    [Total Sales],  
    Sales[Region] = "North"  
)
```

For more complex row-based conditions, FILTER becomes useful.

For example:

Large Order Sales =

```
CALCULATE(  
    [Total Sales],  
    FILTER(  
        Sales,  
        Sales[Quantity] >= 10  
    )  
)
```

For beginners, a useful rule is:

Start with simple CALCULATE filters. Use FILTER when you need more complex filtering logic.

5.7 Removing Filters With REMOVEFILTERS

Sometimes we want a calculation to ignore a particular filter.

Suppose a report contains Region on the rows.

Normally:

Region	Total Sales
North	\$180,000
South	\$290,000
East	\$215,000
West	\$165,000

We can create:

All Region Sales =

```
CALCULATE(  
    [Total Sales],  
    REMOVEFILTERS(Region)  
)
```

```
[Total Sales],  
REMOVEFILTERS(Sales[Region])  
)
```

Now the Region filter is removed for this calculation.

If total company sales are \$850,000, the result could look like:

Region	Total Sales	All Region Sales
North	\$180,000	\$850,000
South	\$290,000	\$850,000
East	\$215,000	\$850,000
West	\$165,000	\$850,000

REMOVEFILTERS is useful when we need to compare a filtered result with a larger total.

5.8 Percentage of Total

A common business calculation is:

What percentage of total company sales came from each region?

First, we already have:

Total Sales =

```
SUM(Sales[Sales Amount])
```

Next:

All Region Sales =

```
CALCULATE(  
    [Total Sales],  
    REMOVEFILTERS(Sales[Region])  
)
```

)

Then:

Sales % of Total =

DIVIDE(

[Total Sales],

[All Region Sales]

)

Suppose:

North Sales = \$170,000

Total Sales = \$850,000

Then:

$170,000 \div 850,000 = 20\%$

So North contributed 20% of total sales.

This example shows how several simple measures can work together.

5.9 ALL

You may also see ALL used in DAX formulas.

For example:

All Region Sales =

CALCULATE(

[Total Sales],

ALL(Sales[Region])

)

In this example, ALL removes the Region filter, producing a result similar to the REMOVEFILTERS example.

For beginners, REMOVEFILTERS is often easier to read when the purpose is simply to remove a filter.

You will encounter ALL more often as you progress into advanced DAX.

5.10 Putting It Together

With CALCULATE, we can turn one basic measure into several useful calculations.

Start with:

Total Sales =

SUM(Sales[Sales Amount])

Then:

North Sales =

CALCULATE(

[Total Sales],

Sales[Region] = "North"

)

Laptop Sales =

CALCULATE(

[Total Sales],

Sales[Product] = "Laptop"

)

High Value Sales =

CALCULATE(

[Total Sales],

FILTER(

Sales,

```
Sales[Sales Amount] > 1000
)
)
```

The important idea is not to memorize these formulas.

Understand the pattern:

Start with a calculation → change or add filters → get a more specific result.

Quick Exercise

Assume you already have:

Total Sales =

SUM(Sales[Sales Amount])

Task 1

Create a measure that calculates sales for the South region.

Task 2

Create a measure that calculates sales for Phones only.

Task 3

Create a measure for transactions where Sales Amount is greater than \$500.

Answers

South Sales =

CALCULATE(

[Total Sales],

Sales[Region] = "South"

)

Phone Sales =

```
CALCULATE(  
    [Total Sales],  
    Sales[Product] = "Phones"  
)
```

Sales Above 500 =

```
CALCULATE(  
    [Total Sales],  
    FILTER(  
        Sales,  
        Sales[Sales Amount] > 500  
    )  
)
```

Chapter Summary

- CALCULATE is used to evaluate a calculation under a modified filter context.
- Simple filters can be added directly inside CALCULATE.
- Multiple filters can be applied at the same time.
- FILTER is useful for more complex filtering conditions.
- REMOVEFILTERS can remove an existing filter.
- ALL can also be used to remove filters in many DAX scenarios.
- CALCULATE becomes much easier to understand once you understand filter context.

A simple way to remember:

CALCULATE = Calculate this, under these conditions.

CHAPTER 7

Time Intelligence & DAX Quick Reference

Many business questions involve time.

For example:

- How much have we sold this year?
- How are sales performing compared with last year?
- What is our year-over-year growth?

DAX provides time intelligence functions to answer these questions.

7.1 Before Using Time Intelligence: Date Tables

For reliable time intelligence, a Power BI model should have a proper Date table.

A simple Date table can be created using:

Date Table =

```
CALENDAR(  
    MIN(Sales[Date]),  
    MAX(Sales[Date])  
)
```

This creates a continuous list of dates covering the sales period.

We can then add useful columns such as:

Year =

```
YEAR('Date Table'[Date])
```

Month =

```
FORMAT('Date Table'[Date], "MMMM")
```

The Date table should be related to the date column in the Sales table:

Date Table[Date] → Sales[Date]

For a proper Power BI model, the Date table should also be marked as a Date table using its Date column.

Key Point: Time intelligence works best with a dedicated, continuous Date table that is correctly related to the data being analyzed.

7.2 Year-to-Date Sales

Year-to-Date (YTD) means:

From the beginning of the year up to the current date or current reporting period.

Suppose we already have:

Total Sales =

SUM(Sales[Sales Amount])

We can create:

Sales YTD =

TOTALYTD(

[Total Sales],

'Date Table'[Date]

)

If our monthly sales are:

Month	Sales	Sales YTD
January	\$10,000	\$10,000
February	\$15,000	\$25,000
March	\$20,000	\$45,000

YTD keeps accumulating sales from the beginning of the year.

7.3 Previous-Year Sales

To compare performance with the previous year, we can create:

Previous Year Sales =

```
CALCULATE(  
    [Total Sales],  
    SAMEPERIODLASTYEAR('Date Table'[Date])  
)
```

Suppose:

2025 Sales = \$800,000

2026 Sales = \$1,000,000

When viewing 2026, [Previous Year Sales] returns the comparable 2025 sales amount.

7.4 Year-over-Year Growth

Once we know current sales and previous-year sales, we can calculate Year-over-Year (YoY) Growth.

The general formula is:

$(\text{Current Sales} - \text{Previous Sales}) \div \text{Previous Sales}$

In DAX:

YoY Growth % =

```
DIVIDE(  
    [Total Sales] - [Previous Year Sales],  
    [Previous Year Sales]  
)
```

Using:

Current Sales = \$1,000,000

Previous Year Sales = \$800,000

we get:

$(1,000,000 - 800,000) \div 800,000$

= 0.25

= 25%

Therefore:

YoY Growth = 25%

Format this measure as a percentage in Power BI.

7.5 Quick DAX Reference

The following table summarizes the main functions covered in this manual.

Function	Purpose
SUM	Adds values
AVERAGE	Calculates an average
MIN	Returns the smallest value
MAX	Returns the largest value
COUNT	Counts numeric values
COUNTA	Counts non-blank values
COUNTROWS	Counts rows in a table
DISTINCTCOUNT	Counts unique values
DIVIDE	Performs safe division

Function	Purpose
CALCULATE	Evaluates a calculation under modified filter context
FILTER	Returns rows that meet a condition
REMOVEFILTERS	Removes specified filters
ALL	Returns all values/rows while ignoring specified filters
TOTALYTD	Calculates a year-to-date value
SAMEPERIODLASTYEAR	Returns the comparable period from the previous year

7.6 Quick Measure Reference

Total Sales

Total Sales =

SUM(Sales[Sales Amount])

Average Sales

Average Sales =

AVERAGE(Sales[Sales Amount])

Total Orders

Total Orders =

DISTINCTCOUNT(Sales[Order ID])

Customer Count

Customer Count =

DISTINCTCOUNT(Sales[Customer ID])

Total Profit

Total Profit =

[Total Sales] - [Total Cost]

Profit Margin

Profit Margin =

DIVIDE([Total Profit], [Total Sales], 0)

Filtered Sales

North Sales =

CALCULATE(

[Total Sales],

Sales[Region] = "North"

)

Year-to-Date Sales

Sales YTD =

TOTALYTD(

[Total Sales],

'Date Table'[Date]

)

Previous-Year Sales

Previous Year Sales =

CALCULATE(

[Total Sales],

SAMEPERIODLASTYEAR('Date Table'[Date])

)

Year-over-Year Growth

YoY Growth % =

DIVIDE(

[Total Sales] - [Previous Year Sales],
[Previous Year Sales]
)

7.7 Final Recap

Throughout this manual, we have learned that:

DAX is the formula language used to create calculations in Power BI.

A calculated column creates a value for each row, while a measure calculates a result based on the current report context.

Basic functions such as:

SUM

AVERAGE

MIN

MAX

COUNTROWS

DISTINCTCOUNT

DIVIDE

allow us to create many everyday business calculations.

Row context deals with the current row, while filter context determines which data is included in a calculation.

CALCULATE allows us to modify that filter context.

Finally, time intelligence allows us to analyze performance across different periods.

The Most Important DAX Principle

Do not focus only on memorizing formulas.

When creating a measure, ask:

1. What am I trying to calculate?
2. Which table and column contain the data I need?
3. Which function performs the calculation?
4. What filters or context should affect the result?

Once you can answer these questions, writing DAX becomes much easier.

OVTech Academy

DAX & Measures in Power BI

Practical DAX for Data Analysis